

Name:G.Sandeep

Hall ticket no:2303A52371

Batch:07

```
def merge_sort(arr):
    """
    Merge Sort Algorithm

    Time Complexity:
    Best Case: O(n log n)
    Average Case: O(n log n)
    Worst Case: O(n log n)

    Space Complexity:
    O(n) auxiliary space

    Sorts the list in ascending order and returns a new sorted list.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test Cases
arr = [38, 27, 43, 3, 9, 82, 10]
print("Original:", arr)
print("Sorted:", merge_sort(arr))
```

Original: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]

```
def binary_search(arr, target):
    """
    Binary Search Algorithm

    Time Complexity:
    Best Case: O(1)
    Average Case: O(log n)
    Worst Case: O(log n)

    Space Complexity:
    O(1)

    Returns index of target if found, else -1.
    """
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
```

```
        else:
            right = mid - 1

    return -1

# Test Cases
sorted_list = [3, 9, 10, 27, 38, 43, 82]
print(binary_search(sorted_list, 27))
print(binary_search(sorted_list, 100))
```

```
3
-1
```

Operation	Recommended Algorithm	Justification
Search by ID	Hash Table (Dictionary)	O(1) average search
Search by Name	Linear Search / Dictionary	Flexible string matching
Sort by Price/Quantity	TimSort (Python sorted())	O(n log n), optimized

```
# Sample Inventory
inventory = [
    {"id": 101, "name": "Laptop", "price": 60000, "quantity": 10},
    {"id": 102, "name": "Mouse", "price": 500, "quantity": 150},
    {"id": 103, "name": "Keyboard", "price": 1500, "quantity": 80},
    {"id": 104, "name": "Monitor", "price": 12000, "quantity": 25}
]

# Fast lookup dictionary
inventory_by_id = {item["id"]: item for item in inventory}

def search_by_id(product_id):
    return inventory_by_id.get(product_id, "Product Not Found")

def search_by_name(name):
    for item in inventory:
        if item["name"].lower() == name.lower():
            return item
    return "Product Not Found"

def sort_by_price():
    return sorted(inventory, key=lambda x: x["price"])

def sort_by_quantity():
    return sorted(inventory, key=lambda x: x["quantity"])

print(search_by_id(102))
print(sort_by_price())
```

```
{'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 150}
[{'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 150}, {'id': 103, 'name': 'Keyboard', 'price': 1500, 'quantity': 80}]
```

Operation	Algorithm	Reason
Search by ID	Hash Table	O(1) lookup
Search by Name	Linear Search	Flexible
Sort by Severity	TimSort	Stable & efficient
Sort by Bill	TimSort	O(n log n)

```
patients = [
    {"id": 1, "name": "Rahul", "severity": 3, "bill": 20000},
    {"id": 2, "name": "Anita", "severity": 5, "bill": 50000},
    {"id": 3, "name": "Kiran", "severity": 2, "bill": 15000}
]

patients_by_id = {p["id"]: p for p in patients}

def search_patient_by_id(pid):
    return patients_by_id.get(pid, "Not Found")

def sort_by_severity():
    return sorted(patients, key=lambda x: x["severity"], reverse=True)

def sort_by_bill():
    return sorted(patients, key=lambda x: x["bill"])

print(sort_by_severity())
```

```
[{'id': 2, 'name': 'Anita', 'severity': 5, 'bill': 50000}, {'id': 1, 'name': 'Rahul', 'severity': 3, 'bill': 20000}, {'id': 3, 'name': 'Kiran', 'severity': 2, 'bill': 15000}]
```

Operation	Algorithm	Justification
Search by Roll No	Binary Search (if sorted) / Dictionary	$O(\log n)$ / $O(1)$
Sort by Marks	Merge Sort / TimSort	$O(n \log n)$

```
students = [
    {"roll": 101, "name": "Asha", "marks": 85},
    {"roll": 102, "name": "Vikram", "marks": 92},
    {"roll": 103, "name": "Neha", "marks": 78}
]

students_by_roll = {s["roll"]: s for s in students}

def search_by_roll(roll):
    return students_by_roll.get(roll, "Not Found")

def sort_by_marks():
    return sorted(students, key=lambda x: x["marks"], reverse=True)

print(sort_by_marks())
```

```
[{'roll': 102, 'name': 'Vikram', 'marks': 92}, {'roll': 101, 'name': 'Asha', 'marks': 85}, {'roll': 103, 'name': 'Neha', 'marks': 78}]
```

Operation	Algorithm	Reason
Search by Order ID	Hash Table	$O(1)$ lookup
Sort by Delivery Time	TimSort	$O(n \log n)$
Sort by Price	TimSort	Efficient

```
orders = [
    {"order_id": 1, "restaurant": "Dominos", "delivery_time": 30, "price": 450},
    {"order_id": 2, "restaurant": "KFC", "delivery_time": 45, "price": 600},
    {"order_id": 3, "restaurant": "Pizza Hut", "delivery_time": 25, "price": 550}
]

orders_by_id = {o["order_id"]: o for o in orders}

def search_order(order_id):
    return orders_by_id.get(order_id, "Not Found")

def sort_by_delivery_time():
    return sorted(orders, key=lambda x: x["delivery_time"])

def sort_by_price():
    return sorted(orders, key=lambda x: x["price"])

print(sort_by_delivery_time())
```

```
[{'order_id': 3, 'restaurant': 'Pizza Hut', 'delivery_time': 25, 'price': 550}, {'order_id': 1, 'restaurant': 'Dominos', 'delivery_time': 30, 'price': 450}, {'order_id': 2, 'restaurant': 'KFC', 'delivery_time': 45, 'price': 600}]
```